RV32IMF 5-MEMBER SPRINT MASTER PLAN

Pipelined RISC-V Processor (CS2202L) | Antigravity-Powered Rapid Execution

GROUP A & B: L1 CACHES | **GROUP C & D: IEEE 754 FPU** | **PERSON E: UART & INTEGRATION** | **GOAL: 220 MINUTES TOTAL**

1. Role Mapping & Feature Ownership

Because Antigravity generates 100% of the Verilog and testbenches, the team's job is **architecture specification, unit testing, and structured integration**. Assign your 5 real names to the roles below:

Team Unit	Assigned Member(s)	Assigned Feature(s)	Primary Deliverables & Scope
Group 1	**Person A & Person B**	Feature 3 (L1 I-Cache) Feature 6 (L1 D-Cache)	`icache.v`, `dcache.v`, tag comparison, hit/miss FSM, 4-word refill, cache stalls.
Group 2	**Person C & Person D**	Feature 4 (IEEE 754 FPU)	`fpu_unit.v` (FADD.S, FSUB.S, FMUL.S), `fpu_regfile.v` (f0-f31), single-precision arithmetic.
Lead	**Person E**	UART MMIO & Master Verification Lead	`uart_tx.v` (115200 baud at `0x80000000`), Git master branch, merges, and demo test scripts.

> **The Non-Negotiable Success Rule:** *Zero "Big-Bang" Merging!* Never wait until the end of the day to merge everything together. You will merge in **3 incremental stages** (UART first → FPU second → Caches third). If a feature has a bug, the rest of the processor stays 100% functional!

2. High-Efficiency Parallel Timeline (In Minutes)

Total time is compressed to **~220 minutes (~3.5 hours)** because teammates don't type manual RTL. Each phase runs in parallel:

Elapsed Time	Group A & B (Caches)	Group C & D (IEEE FPU)	Person E (UART & Lead)
T + 0 to 40 min *(Phase 1: Standalone)*	Prompt Antigravity for `icache.v` & `dcache.v` + testbenches `tb_icache.v`, `tb_dcache.v`.	Prompt Antigravity for `fpu_unit.v` & `fpu_regfile.v` + testbench `tb_fpu.v`.	Prompt Antigravity for `uart_tx.v`. Wire to base core at MMIO `0x80000000`.
T + 40 to 75 min *(Phase 2: Unit Sim)*	Run Vivado simulations on `tb_icache` & `tb_dcache`. Verify 1-cycle hit & multi-cycle miss stall.	Run Vivado simulation on `tb_fpu`. Verify 100+ vectors for addition, subtraction, multiplication.	Run test on base core + UART. **VERIFY:** Core prints `"HELLO RISCV"` over serial!
T + 75 to 115 min *(Phase 3:*	Document cache hit rate & latency metrics; review memory bus arbiter interface.	🤝 **PERSON E MERGES WITH C & D (FPU INTEGRATION):** Wire FPU into `execute_stage.v`. Run assembly test `2.5 + 3.75 = 6.25` and print result to UART. **100% Working!**	

Elapsed Time	Group A & B (Caches)	Group C & D (IEEE FPU)	Person E (UART & Lead)
CHECKPOINT 1)			
T + 115 to 165 min *(Phase 4: CHECKPOINT 2)*	🤝 **PERSON E MERGES WITH PERSON A (I-CACHE INTEGRATION):** Insert `icache.v` between Fetch (IF) stage and instruction memory. Verify core fetches instructions through cache and continues printing to UART.		Supervises pull request and runs regression suite.
T + 165 to 210 min *(Phase 5: CHECKPOINT 3)*	🤝 **PERSON E MERGES WITH PERSON B (D-CACHE INTEGRATION):** Insert `dcache.v` into Memory (MEM) stage. Run load/store stress test and floating point benchmark.		Checks timing slack in Vivado. Confirms zero regressions.
T + 210 to 220 min *(Wrap-up)*	🎉 **FINAL COMMIT & PUSH TO GITHUB MAIN. 100% PROJECT COMPLETE & CALL THE DAY OFF!**		

3. Exact Workflow for Person E (UART & Verification Lead)

> **Mission:** Build the test harness that proves everything works. By minute 75, you will have the core printing messages over UART to your terminal. Then you lead the 3 checkpoints.

Step E1: Clone & Create Your Branch (Minute 0)

Open VS Code / Terminal in your workspace:

```
git checkout main
git pull origin main
git checkout -b feature-uart
```

Step E2: Feed Prompt to Antigravity (Minute 5)

Copy and paste this prompt into Antigravity:

> *"Create a synthesizable 115200-baud UART transmitter module `uart_tx.v` (8 data bits, 1 stop bit, no parity) for a 100 MHz clock. Integrate it into `pipeline_5stage.v` as a memory-mapped peripheral: whenever an instruction writes a byte via Store Word/Byte to address `0x80000000`, the byte is pushed into the UART TX FIFO and transmitted on FPGA pin `D18` (Nexys A7 UART TX)."*

Step E3: Verify UART on Base Core (Minute 45)

Assemble and run this simple 3-line RISC-V test program:

```
li x1, 0x80000000   # UART MMIO Address
li x2, 65           # ASCII 'A'
sw x2, 0(x1)        # Transmit 'A'!
```

Open PuTTY or TeraTerm (Baud 115200, COM port). Verify that character **'A'** or **"HELLO"** appears on screen! Once this works, you are ready to verify everyone else's features.

---

4. Exact Workflow for Group C & D (IEEE 754 FPU)

> **Mission:** Implement single-precision 32-bit floating point arithmetic (FADD.S, FSUB.S, FMUL.S) and 32 FP registers (f0–f31).

Step CD1: Clone & Create Your Branch (Minute 0)

```
git checkout main
git pull origin main
git checkout -b feature-fpu
```

Step CD2: Feed Prompt to Antigravity (Minute 5)

> *"Implement an IEEE 754 single-precision 32-bit Floating Point Unit in `feature_4/modules/fpu_unit.v`. Support operations: FADD.S (funct7=0000000), FSUB.S (funct7=0000100), and FMUL.S (funct7=0001000). Also generate a 32x32-bit floating point register file `fpu_regfile.v` with 2 read ports and 1 write port. Create a self-checking testbench `tb_fpu.v` verifying 100 directed and randomized test vectors (e.g. 1.0 + 2.0 = 3.0, 2.5 * 4.0 = 10.0, negative numbers, and zero)."*

Step CD3: Standalone Simulation in Vivado (Minute 40)

Run the testbench inside Vivado:

```
xvlog -sv feature_4/modules/fpu_unit.v feature_4/modules/fpu_regfile.v feature_4/tb_fpu.v
xelab -debug typical tb_fpu -s sim_fpu
xsim sim_fpu -R
```

Success Criterion: `100% (100/100) Passed with 0 Errors`.

Step CD4: Merge Checkpoint 1 with Person E (Minute 80)

Connect FPU into the execution stage by making these 3 simple edits:

1. In `decode_stage.v`: Add `wire is_fpu = (opcode == 7'b1010011);`
2. In `execute_stage.v`: Instantiate `fpu_unit u_fpu (...)`
3. In the EX result MUX: Route `fpu_result` when `is_fpu` is high.

Person E runs a program computing `2.5 + 3.75 = 6.25` and prints the IEEE float hex bits (`0x40C80000`) to UART.
Checkpoint 1 PASSED!

---

5. Exact Workflow for Group A & B (L1 I-Cache & D-Cache)

> **Mission:** Eliminate memory bottlenecks by building a direct-mapped Instruction Cache (Person A) and Data Cache (Person B).

Step AB1: Clone & Create Your Branch (Minute 0)

```
git checkout main
git pull origin main
git checkout -b feature-cache
```

Step AB2: Division of Labor between A & B

- **Person A focuses on `icache.v`:** Read-only, single-cycle hit latency, 4-word block refill FSM, outputs `icache_stall`.
- **Person B focuses on `dcache.v`:** Read/Write, write-through policy, word/byte alignment (SB, SH, SW), outputs `dcache_stall`.

Step AB3: Feed Prompt to Antigravity (Minute 5)

> *"Implement a direct-mapped L1 Instruction Cache in `feature_3/modules/icache.v` with 1 KB capacity (64 lines of 16 bytes / 4 words each). Implement tag array, valid bit array, single-cycle hit comparison, and a 4-cycle burst refill FSM that asserts `icache_stall` on a miss. Also generate `feature_6/modules/dcache.v` with write-through policy for data memory. Include standalone testbenches `tb_icache.v` and `tb_dcache.v`."*

Step AB4: Standalone Simulation in Vivado (Minute 45)

```
xvlog -sv feature_3/modules/icache.v feature_3/tb_icache.v
xelab -debug typical tb_icache -s sim_icache
xsim sim_icache -R
```

Success Criterion: Verify that consecutive fetches to the same block report `HIT (0 stall cycles)`, and initial fetches report `MISS (asserts stall for refill)`.

Step AB5: Merge Checkpoint 2 (I-Cache) with Person E (Minute 120)

1. Place `icache.v` between Fetch (IF) stage and memory bus.
2. Connect `icache_stall` to `hazard_unit.v` to freeze the PC and IF/ID register during a refill.
3. Person E runs the UART print test. If text prints cleanly, **I-Cache integration is 100% successful!**

Step AB6: Merge Checkpoint 3 (D-Cache) with Person E (Minute 170)

1. Place `dcache.v` in the Memory (MEM) stage.
2. Connect `dcache_stall` to freeze the entire pipeline during data miss refills.
3. Run full test suite: Multiplication + FPU Math + Load/Store + UART.

---

6. Staged Integration Checkpoints Summary

Checkpoint	Target Time	Modules Integrated	Verification Proof
Checkpoint 1	**Minute 115**	5-Stage Core + Multiplier + **UART + FPU**	Float multiplication printed on terminal: `2.5 * 4.0 = 10.0`.
Checkpoint 2	**Minute 165**	Previous Core + **L1 I-Cache**	Instructions execute from cache; hit rate logged; UART outputs.
Checkpoint 3	**Minute 210**	Full Core + **L1 D-Cache** (All Features Unified)	Full C benchmark suite passes (`run_c_tests.py all`).

7. Git Collaboration & Conflict Resolution (For LKG Kids)

Follow these exact terminal commands step-by-step. Do not improvise:

How to Push Your Completed Feature Branch

When your standalone feature passes tests, push your branch to GitHub:

```
# Check what files you modified
git status

# Stage your files
git add .

# Commit with a clear message
git commit -m "Completed and verified Feature standalone tests"

# Push to your remote branch
git push origin <your-branch-name>
```

How Person E Merges into Main

```
# Switch to main and get latest
git checkout main
git pull origin main

# Merge the feature branch
git merge feature-fpu
```

What if Git Shows a MERGE CONFLICT? (Zero Panic!)

If two people modified `execute_stage.v`, Git will flag a conflict. **Do not panic.** Simply ask Antigravity:

> **Prompt to Antigravity:**
> *"Resolve the git merge conflict in `execute_stage.v` between `main` and branch `feature-fpu`. Preserve both the `multiplier_unit` result multiplexer and the new `fpu_unit` result multiplexer so both RV32M and RV32F instructions work seamlessly."*

Antigravity will instantly resolve the conflict cleanly in 5 seconds.

---

8. Emergency Troubleshooting Matrix

Symptom / Error	Probable Root Cause	Exact Instant Fix
Core hangs / freezes on first instruction	`icache_stall` is permanently stuck high because `valid_bit` array was uninitialized.	In `icache.v`, add an active-low reset loop initializing all `valid[i] <= 1'b0;` on `reset_n == 0`.
UART outputs weird / garbled symbols	Baud rate clock divider mismatch (e.g. 50 MHz divider used instead of 100 MHz).	For 100 MHz clock and 115200 baud, set clock divider parameter to exactly `868` (100,000,000 / 115200).

Symptom / Error	Probable Root Cause	Exact Instant Fix
FPU produces NaN or 0 on valid inputs	Significand hidden leading bit (`1.M`) was omitted during normalization.	Ensure mantissa is expanded to 24 bits: `{1'b1, fraction[22:0]}` for normalized numbers.
Negative slack in Vivado after adding FPU	FPU combinational path + ALU multiplexer exceeds clock period.	Ensure target clock constraint in `nexys_a7_100t.xdc` is set to **20.0 ns (50 MHz)**.

Ready for Execution!

Map your real names to Roles A, B, C, D, E • Follow the Minute Milestones • Enjoy a seamless, victorious demo!